

University Of Asia Pacific

Department of Computer Science and Engineering

CSE 402:Operating System Lab

LAB REPORT

Assignment Number: 1
Assignment Title: Implementing FCFS CPU Scheduling

Submitted by:
Name:Md. Muhaimin Al Baki Raiyan
Student ID:23101163
Section:C2

Submitted To:
Atia Rahman Orthi
Lecturer,
Department of Computer Science and Engineering

Date of Submission: 02.08.2026

FCFS CPU Scheduling — Lab Report

CSE 402 — Operating System Lab

1. Objective

To implement the First Come First Serve (FCFS) CPU scheduling algorithm for a given set of 5 processes, and to compute the Completion Time (CT), Turnaround Time (TAT), Waiting Time (WT), Average TAT, Average WT, and the process execution sequence.

2. Problem Statement

Implement FCFS CPU Scheduling for the following scenario where the number of processes = 5, with the given Arrival Time (AT) and Burst Time (BT) for each process. Calculate CT, TAT, WT, Average TAT and Average WT, and represent the process execution sequence.

P id	AT	BT
P0	3	1
P1	5	3
P2	2	2
P3	1	2
P4	6	3

3. Algorithm

- Sort all processes in ascending order of their Arrival Time (AT).
- Initialize the current system time to 0.
- For each process in the sorted order: if the CPU is idle (current time < AT of process), advance the time to the process's AT.
- Add the process's Burst Time (BT) to the current time to get its Completion Time (CT).
- Compute Turnaround Time: TAT = CT − AT.
- Compute Waiting Time: WT = TAT − BT.
- Repeat for all processes and record the execution sequence.
- Compute Average TAT and Average WT by dividing the totals by the number of processes.

4. Source Code (Python)

```
processes = [
    {"id": "P0", "AT": 3, "BT": 1},
    {"id": "P1", "AT": 5, "BT": 3},
    {"id": "P2", "AT": 2, "BT": 2},
    {"id": "P3", "AT": 1, "BT": 2},
    {"id": "P4", "AT": 6, "BT": 3},
]

processes.sort(key=lambda p: p["AT"])

time = 0
sequence = []
print(f"{'Pid':<5}{'AT':<5}{'BT':<5}{'CT':<5}{'TAT':<5}{'WT':<5}")
total_tat = 0
total_wt = 0
for p in processes:
    if time < p["AT"]:
        time = p["AT"]
    time += p["BT"]
    ct = time
    tat = ct - p["AT"]
    wt = tat - p["BT"]
    total_tat += tat
    total_wt += wt
    sequence.append(p["id"])
    print(f"{p['id']:<5}{p['AT']:<5}{p['BT']:<5}{ct:<5}{tat:<5}{wt:<5}")

n = len(processes)
print(f"\nAverage TAT = {total_tat/n:.2f}")
print(f"Average WT = {total_wt/n:.2f}")
print(f"\nExecution sequence: {' -> '.join(sequence)}")
```

5. Output

```
Pid  AT   BT   CT   TAT  WT
P3   1    2    3    2    0
P2   2    2    5    3    1
P0   3    1    6    3    2
P1   5    3    9    4    1
P4   6    3    12   6    3

Average TAT = 3.60
Average WT = 1.40

Execution sequence: P3 -> P2 -> P0 -> P1 -> P4
```

6. Result Table

Pid	AT	BT	CT	TAT	WT
P3	1	2	3	2	0
P2	2	2	5	3	1
P0	3	1	6	3	2
P1	5	3	9	4	1
P4	6	3	12	6	3

7. Gantt Chart (Execution Sequence)

	P3	P2	P0	P1	P4
0 - 1	1 - 3	3 - 5	5 - 6	6 - 9	9 - 12

Execution Sequence: P3 → P2 → P0 → P1 → P4

8. Calculations

Process P3 (AT = 1, BT = 2)

CT = max(Previous CT, AT) + BT = max(0, 1) + 2 = 1 + 2 = 3

TAT = CT − AT = 3 − 1 = 2

WT = TAT − BT = 2 − 2 = 0

Process P2 (AT = 2, BT = 2)

CT = max(Previous CT, AT) + BT = max(3, 2) + 2 = 3 + 2 = 5

TAT = CT − AT = 5 − 2 = 3

WT = TAT − BT = 3 − 2 = 1

Process P0 (AT = 3, BT = 1)

CT = max(Previous CT, AT) + BT = max(5, 3) + 1 = 5 + 1 = 6

TAT = CT − AT = 6 − 3 = 3

WT = TAT − BT = 3 − 1 = 2

Process P1 (AT = 5, BT = 3)

CT = max(Previous CT, AT) + BT = max(6, 5) + 3 = 6 + 3 = 9

TAT = CT − AT = 9 − 5 = 4

WT = TAT − BT = 4 − 3 = 1

Process P4 (AT = 6, BT = 3)

CT = max(Previous CT, AT) + BT = max(9, 6) + 3 = 9 + 3 = 12

TAT = CT − AT = 12 − 6 = 6

WT = TAT − BT = 6 − 3 = 3

Averages

Average Turnaround Time (TAT) = (2 + 3 + 3 + 4 + 6) / 5 = 18 / 5 = 3.60

Average Waiting Time (WT) = (0 + 1 + 2 + 1 + 3) / 5 = 7 / 5 = 1.40

9. Discussion

The FCFS algorithm allocates the CPU on a non-preemptive, arrival-order basis — once a process is picked up, it runs to completion, and the only factor that decides who goes next is who showed up first. Burst time plays no role whatsoever in the ordering decision. This is exactly what is reflected in the simulation: even though the five processes were originally listed as P0 through P4, sorting them by Arrival Time produced the sequence P3, P2, P0, P1, P4, since P3 arrived earliest at time 1 and P4 arrived last at time 6.

At the very start of the timeline, the CPU sits idle from time 0 to time 1, because no process has arrived yet — P3, the earliest arrival, only enters the ready queue at AT = 1. This idle interval is visible as the blank slot at the beginning of the Gantt chart in Section 7. From that point onward, the schedule runs back-to-back with no further gaps: P2 arrives at time 2 while P3 is still executing, so it simply waits its

turn; by the time each subsequent process is considered, its arrival time has already passed, so the CPU never has to idle again.

Examining the waiting times reveals the central weakness of this approach. P3, which happens to have both an early arrival and a short burst, finishes with zero waiting time — a best-case outcome. P0, on the other hand, needs only 1 unit of CPU time to finish, yet it is forced to sit in the queue while P2 (which needs 2 units) is served first, purely because P2 arrived one unit earlier. This is the classic convoy effect: a short job stuck queued behind a longer one, inflating the waiting time of processes that would otherwise finish almost instantly. P4 suffers the most, waiting 3 units, mainly because it is last in arrival order and inherits the cumulative delay of every process ahead of it.

Overall, the schedule produced an Average Turnaround Time of 3.60 units and an Average Waiting Time of 1.40 units. These figures are respectable mainly because the workload in this particular scenario has low burst times and closely spaced arrivals; had a long process arrived early, every later process would have been forced to wait considerably longer, dragging both averages up significantly. This sensitivity to arrival order — rather than to job length — is what distinguishes FCFS from priority-aware algorithms such as SJF or SRTF, which would have reordered execution to let P0 run ahead of P2 and reduced the overall waiting time.

10. Conclusion

This experiment successfully demonstrates the working principle of the First Come First Serve CPU scheduling algorithm through a Python implementation applied to a set of 5 processes with distinct arrival and burst times. By sorting the processes on Arrival Time and simulating the CPU allocation step by step, the program correctly computed the Completion Time, Turnaround Time, and Waiting Time for every process, along with an Average Turnaround Time of 3.60 units and an Average Waiting Time of 1.40 units.

The resulting execution order, P3 → P2 → P0 → P1 → P4, along with the brief CPU idle period from time 0 to 1 captured in the Gantt chart, confirms that FCFS is driven entirely by arrival sequence rather than process length or priority. This behaviour was further reinforced by the discussion, which highlighted how a shorter job like P0 can still be delayed behind an earlier-arriving but longer job like P2 — the convoy effect — leading to higher average waiting times than would be achievable under a burst-time-aware policy such as SJF or SRTF.

Overall, the lab reinforces the theoretical understanding of non-preemptive scheduling: FCFS is simple to implement and completely fair in terms of arrival order, but it does not optimize for throughput or minimize waiting time, since it takes no account of how long each process actually needs the CPU. This trade-off between simplicity and efficiency is precisely why operating systems typically favor more

adaptive algorithms — such as SJF, Priority Scheduling, or Round Robin — in scenarios where reducing average waiting and turnaround time is a priority.